User Manual

Coding Notebook Sheets App

Day selector, multi-sheet notebook, JSON storage, and built-in Python runner

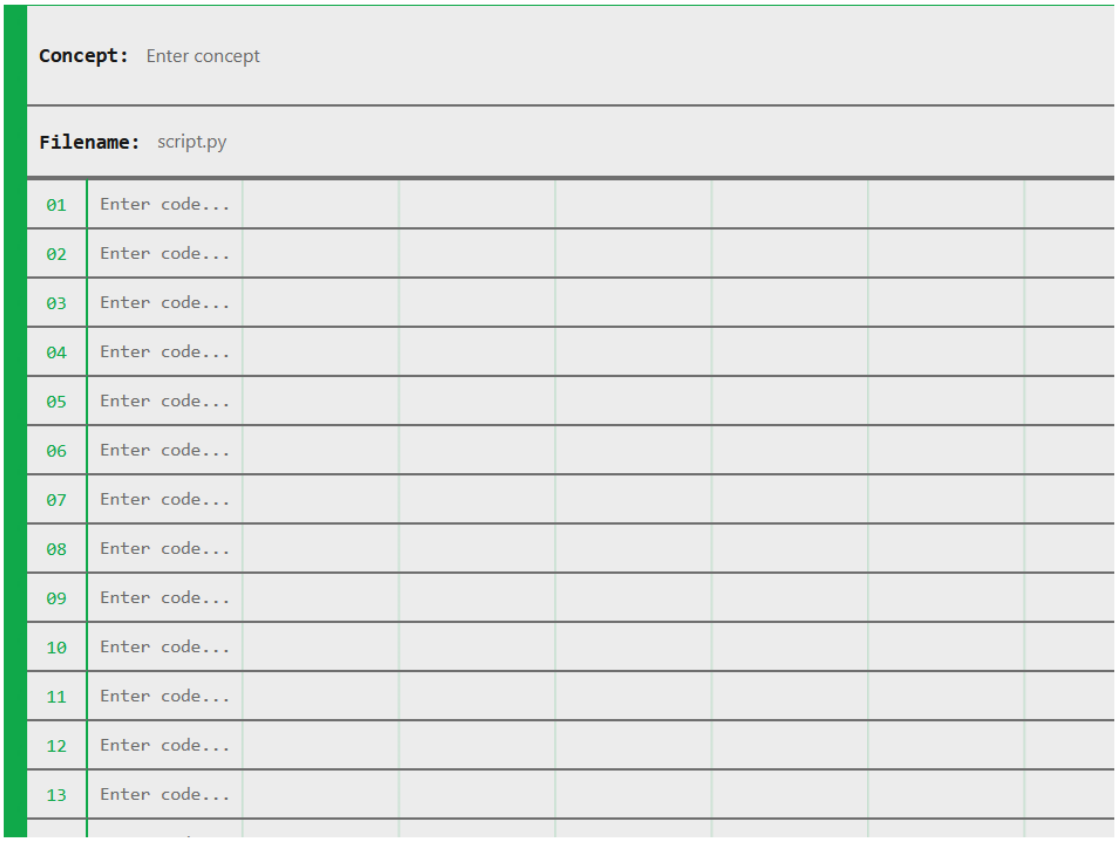

Figure 1. Main notebook sheet interface.

Prepared for application users and reviewers

Version: Revised edition reflecting day selection, multi-sheet storage, repositioned runner controls, and the latest sheet-entry behavior

Purpose: Step-by-step instructions for creating, saving, navigating, importing, exporting, and running code in the application

Table of Contents

1. Introduction

2. System requirements and file package

3. Getting started

4. Understanding the interface

5. Working with days and sheets

6. Entering and organizing notebook content

7. Saving, autosave, and data storage

8. Importing and exporting JSON notebook files

9. Using the Python runner

10. Recommended workflows

11. Troubleshooting and recovery

12. Best practices

13. Appendix: notebook JSON structure

1. Introduction

The Coding Notebook Sheets App is a browser-based notebook system designed around the printed coding notebook sheets supplied with the application. The digital interface keeps the same core sheet logic—concept, filename, numbered code lines, comments, summary, lecture, and day tracking—while adding storage, search, navigation, and code execution features.

The current version supports a notebook model in which a single JSON file can contain many days, and each day can contain more than one sheet. This makes the app practical for daily lessons, labs, projects, and multi-page coding exercises.

The application is intended for learners, instructors, reviewers, and independent users who want to keep line-by-line coding notes in a structured notebook rather than in scattered files.

The source paper template remains the design reference for the interface layout, which helps users transition easily between the printed notebook sheet and the digital entry form.

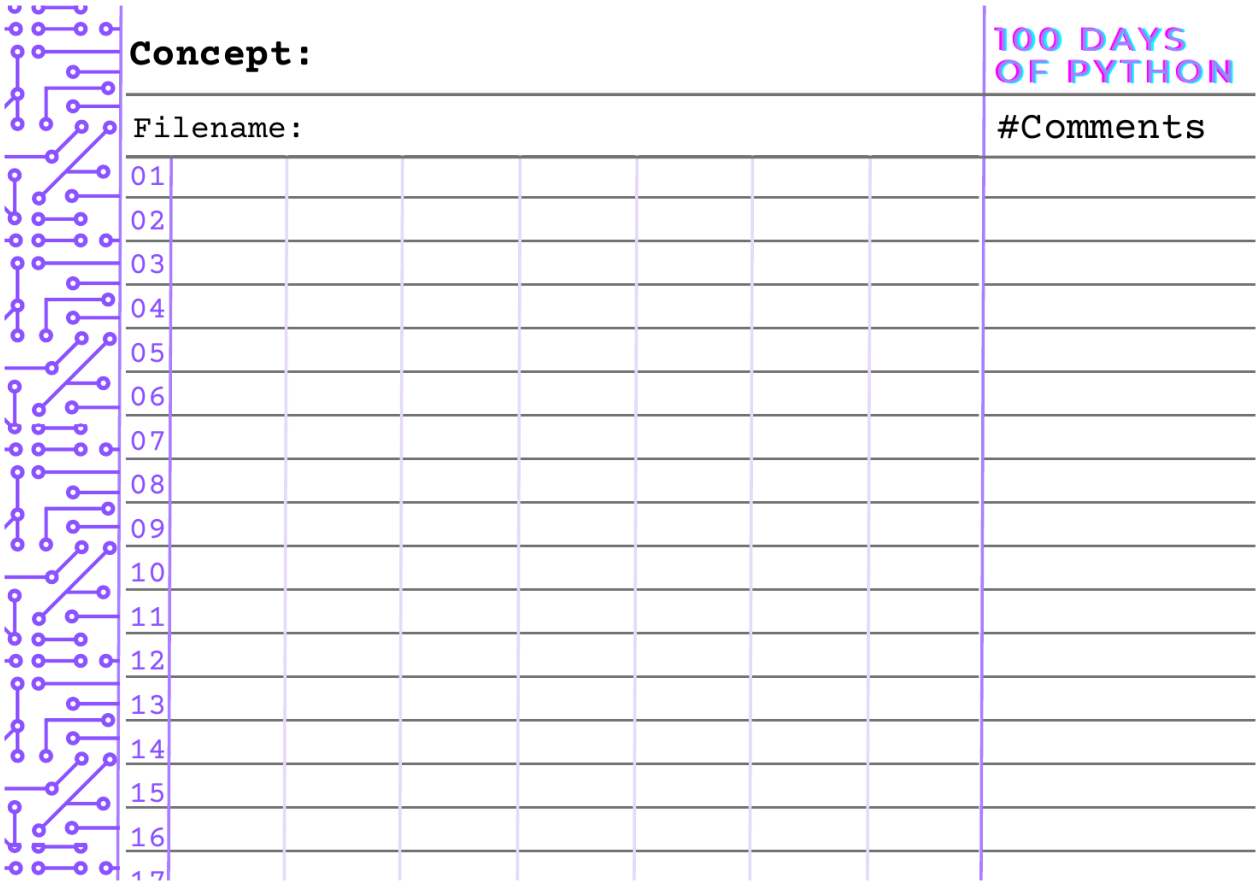

Figure 2. Source notebook template used as the visual model for the digital sheet.

Primary capabilities

- Create and edit notebook sheets for any day.
- Store multiple sheets under the same day.
- Move between saved sheets without losing content.
- Select a day directly instead of relying only on sequential navigation.
- Save all notebook content in the browser and export it to one JSON notebook file.
- Import a previously exported notebook file and continue working.
- Load code lines 01-18 into the Python runner and execute them in the browser.
- Clear the runner input and output areas separately.
- Keep sheet entry closely aligned to the original printed notebook structure.

2. System requirements and file package

To use the application successfully, make sure the following conditions are met.

- A modern web browser such as Chrome, Edge, or Firefox.
- JavaScript enabled in the browser.
- Internet access on first load if you want to use the Python runner, because the runner loads the Pyodide runtime from a network source.
- Permission to download and upload JSON files if you plan to export or import notebook data.

Package contents

- index.html - the main application page.
- styles.css - the interface styling file.
- app.js - the application logic file.
- sample_notebook.json - an example notebook data file.
- README or text documentation files included with the package.

Important storage note

The app stores notebook entries in two places, depending on your actions. First, it uses browser local storage for immediate persistence on the same device and browser. Second, it allows you to export the entire notebook collection to one JSON file for backup, transfer, or archival use.

3. Getting started

Open the application

- Extract the ZIP package to a folder on your computer.
- Open the application folder.
- Double-click index.html, or right-click it and open it in your preferred browser.
- Wait for the interface to load. On first launch, the app creates an initial day and sheet if no notebook exists yet.

What you see on first launch

The interface is divided into two main areas: a left sidebar and a main notebook workspace. The sidebar contains day controls, stored sheet navigation, data storage actions, and search tools. The main workspace contains the entry sheet and the Python runner placed below it.

First-run checklist

- Confirm that the day selector shows a day value, usually Day 1.
- Confirm that the sheet status line shows the currently selected day and sheet.
- Type a concept name and a filename to confirm that the form fields are active.
- Enter a sample line of code in line 01 and verify that the autosave status updates if autosave is enabled.

4. Understanding the interface

Top action buttons

The top action area provides fast access to common sheet-level actions such as creating a new sheet, saving the current sheet, deleting a sheet, or moving through sheets for the selected day.

Sidebar: Days and Stored Sheets

- Select Day dropdown: changes the active day context.
- Add Day: creates a new day if it does not already exist.
- Go to Day: jumps directly to a specified day.
- Search box: filters the sheet list for the selected day.
- Stored sheet cards: show the concept title, day number, sheet number, and filename for each saved sheet.

Sidebar: Data Storage

- Export Full Notebook JSON: downloads all days and all sheets in one file.
- Import JSON: loads a previously saved notebook file into the app.
- Download Current Sheet JSON: saves only the active sheet as its own JSON file.
- Duplicate Current Sheet: creates a copy of the active sheet and assigns it the next sheet number within that day.

Main notebook workspace

- Concept: the lesson topic, exercise title, or coding focus.
- Filename: the program or script name connected to the current sheet.
- Code rows 01-18: numbered entry lines used for line-by-line code entry.
- Comments: notes, explanations, reminders, or debugging observations.
- Summary: a short record of what was learned or completed.
- Lecture: the class, lesson, or reference associated with the sheet.

Runner area under the entry sheet

- Input window: receives code copied or loaded from the numbered code rows.
- Output window: displays execution results, messages, or error text.
- Clear Input: removes text from the runner input window.
- Clear Output: removes text from the runner output window.
- Run Python: executes the code currently loaded in the runner input area.

5. Working with days and sheets

Day selection

The current application version lets you select a day directly. This is useful when you need multiple sheets for one day or want to move back to earlier days without stepping through every intermediate page.

- Use the day selector to choose an existing day.
- Use Add Day to create a new day record.
- Use Go to Day to switch to the selected day.
- A day can contain one or many sheets.

Multiple sheets within the same day

You can create multiple sheets for one day when a lesson, lab, or project is too long for a single page.

- Create Sheet or New Sheet adds another sheet under the same day.
- Each sheet receives its own sheet number within that day.
- Previous Sheet and Next Sheet let you move through the sheets for the selected day.

Navigating the notebook

- Use the day controls to move between daily groups of sheets.
- Use the sheet cards to open a specific saved entry.
- Use Duplicate Current Sheet when you want to reuse a structure and edit it into a new version.

6. Entering and organizing notebook content

Concept field

Use the Concept field for the topic, chapter, lesson, or project title. Clear concept names make stored sheets easier to search later.

Filename field

Use the Filename field for the script or program file related to the code on the sheet. The filename area is aligned to the same ruled structure used by the comments section so the visual flow stays consistent with the sheet design.

Code entry rows

The numbered code rows are designed to be entered line by line.

- Enter one code line per numbered row whenever possible.
- Press Enter in a code row to move to the next code line more naturally.
- Use blank lines only when needed to separate logical sections of code.

Comments section

The comments section is intended for explanations, debugging notes, reminders, or step-by-step observations. The line spacing in this area has been adjusted so typed text fits more cleanly between the ruled lines.

- Use comments to explain what a block of code does.
- Use comments to record errors, tests, or issues found during coding.
- Press Enter in the comments area to continue to a new line naturally.

Summary and lecture fields

Write a brief record of what the sheet covers in the Summary field. Use the Lecture field to note the lesson name, course source, or related instructional reference.

7. Saving, autosave, and data storage

Save behavior

The application can save notebook entries manually and, when enabled, automatically. Manual save controls may be useful before export or before major editing changes.

- Browser local storage preserves the current notebook on the same device and browser.
- Each stored sheet keeps its day number, sheet number, concept, filename, code lines, comments, summary, lecture, and timestamps used by the app.

Autosave

If autosave is enabled in the application, edits are written to storage while you work. Manual Save remains useful when you want an explicit save point before duplication, import, export, or deletion.

Single-file notebook model

One of the major advantages of the current design is that many days and many sheets can be preserved in one shared notebook structure instead of being scattered across separate files.

8. Importing and exporting JSON notebook files

Exporting the full notebook

- Use Export Full Notebook JSON to download all days and all sheets in one file.
- Store exported files in a clearly named folder so you can identify notebook versions later.

Downloading the current sheet

If you need only the active entry, use Download Current Sheet JSON. This is useful for sharing a single exercise without sending the entire notebook.

Importing a notebook

- Use Import JSON to restore a previously exported notebook file.
- After import, verify that the correct day and sheet appear before continuing to edit.

9. Using the Python runner

Loading code into the runner

- Enter code on lines 01-18 of the notebook sheet.
- Click Load 01-18 into Runner to move those lines into the runner input area.
- Review the code in the input panel before execution.

Running code

- Click Run Python to execute the current code in the runner input area.
- Read results, print output, or errors in the output window below.
- Use Clear Output when you need to remove prior results before another test.
- Use Clear Input when you want to remove existing runner text without changing the notebook sheet itself.

Practical note

The Python runner uses a browser-based runtime. On first use, allow the runtime to finish loading before you run code. If the runtime does not start, refresh the page and check that network access is available.

10. Recommended workflows

Daily study workflow

- Select the study day.
- Create a new sheet if needed.
- Enter the concept and filename first.
- Fill in code rows, comments, summary, and lecture.
- Run code in the runner area.
- Export the notebook at the end of the session.

Multi-page lesson workflow

- Choose the day for the lesson.
- Use new sheet creation to add another page when the code or notes exceed one sheet.

- Keep sheet numbering in sequence and use consistent concept names so related sheets are easy to identify.

Review workflow

- Open an earlier day from the selector.
- Review each sheet in order.
- Update comments with corrected explanations or debugging notes.
- Re-run code in the runner when you need to verify behavior.

11. Troubleshooting and recovery

Common issues and responses

- If the runner does not execute, refresh the page and wait for the browser-based Python runtime to finish loading.
- If a day appears empty, confirm that you are on the correct selected day and check whether more than one sheet exists for that day.
- If imported data does not appear immediately, verify that the JSON file matches the notebook structure expected by the application.
- If text appears crowded in the comments area, press Enter normally rather than pasting long unbroken paragraphs.
- If a code line appears in the wrong row, move through the code rows with Enter so the line sequence stays aligned to the numbered sheet.

Recovery advice

Keep regular JSON exports of the notebook. Browser local storage is useful for convenience, but exported JSON files are the safest backup method for long-term retention and transfer to another device.

12. Best practices

- Export your notebook regularly as a backup.
- Use clear filenames that match your actual scripts.
- Keep concept names meaningful so stored sheets remain easy to search.
- Use the summary field to capture the purpose of each sheet.
- Use the comments area for reasoning, not just copied code.
- Keep one major topic per sheet whenever possible.
- Use multiple sheets for the same day instead of crowding one page.

13. Appendix: notebook JSON structure

The exact structure used by the application may vary by version, but the overall notebook is expected to include a notebook-level object with nested day and sheet records. A simplified example is shown below.

```
{
  "days": {
    "1": [
      {
        "sheetNumber": 1,
        "concept": "Loops and Lists",
        "filename": "loops_lesson.py",
        "codeLines": [
          "items = [1, 2, 3]",
          "for item in items:",
          "    print(item)"
        ],
        "comments": "Basic loop practice.",
        "summary": "Reviewed for-loops and list iteration.",
        "lecture": "Python Day 12"
      }
    ],
    "2": [
      {
        "sheetNumber": 1,
        "concept": "Functions",
        "filename": "functions_lab.py"
      }
    ]
  }
}
```


Closing Note

The Coding Notebook Sheets App is designed to bridge printed study habits and digital notebook storage. When used consistently, it becomes a practical record of coding progress, lesson notes, and runnable examples that can be searched, exported, and reviewed over time.